Experiment No. 5

Aim

To perform hands-on Solidity programming assignments for creating smart contracts and understand Solidity program structure, basic constructs, compilation and deployment using Remix IDE.

Theory

Solidity is a high-level, object-oriented programming language used to develop smart contracts on the Ethereum blockchain. Smart contracts are programs stored and executed on the blockchain. Remix IDE provides an environment to write, compile, deploy and execute Solidity smart contracts.

1. Introduction and Basic Syntax

A Solidity program generally consists of a license identifier, compiler version declaration, contract definition, state variables and functions.

The pragma statement specifies the compatible compiler version, while the contract keyword is used to define a smart contract.

2. Primitive Data Types

Solidity provides different primitive data types:

- **uint / int:** Used to store unsigned and signed integers.
- **bool:** Stores either true or false.
- **address:** Stores a 20-byte Ethereum address.
- **bytes:** Stores fixed-size or dynamic binary data.
- **string:** Stores textual information.

3. Variables

Variables in Solidity are classified into three types:

- **State Variables:** Declared inside a contract and stored permanently on the blockchain.
- **Local Variables:** Declared inside functions and used temporarily during execution.
- **Global Variables:** Special predefined variables such as msg.sender, msg.value and block.timestamp.

4. Functions

Functions contain the instructions that define the behaviour of a smart contract.

Reading and Writing to a State Variable: Functions can read the values of state variables and modify them. Writing to a state variable changes the contract's blockchain storage.

View and Pure Functions:

- view: Allows reading state variables but does not modify them.
- pure: Does not read or modify contract state and performs calculations using inputs.

Modifiers and Constructors:

- **Modifiers:** Reusable code blocks used to change or restrict function execution, such as checking user permissions.
- **Constructors:** Special functions executed only once during contract deployment to initialize values.

Inputs and Outputs: Functions can accept parameters as inputs and return one or more values as outputs. This allows data to be passed into functions and results to be returned.

5. Visibility

Visibility determines how functions and state variables can be accessed.

- **Public:** Accessible from within and outside the contract.
- **Private:** Accessible only within the contract in which they are declared.
- **Internal:** Accessible within the contract and its derived contracts.
- **External:** Intended to be called from outside the contract.

6. Control Flow

Control flow determines the order in which statements are executed.

If-Else: Used to execute different statements depending on whether a condition is true or false.

Loops: Used to execute a block of code repeatedly. Solidity supports for, while and do-while loops. Loops must be used carefully because they consume gas during execution.

7. Data Structures

- **Arrays:** Used to store ordered collections of elements. They can be fixed-size or dynamic.
- **Mappings:** Store data in key-value pairs and provide efficient lookup.
- **Structs:** Allow multiple related variables of different data types to be grouped together.
- **Enums:** Define a set of predefined named values, improving code readability.

8. Data Locations

Solidity uses three primary data locations:

- **Storage:** Stores data permanently on the blockchain.
- **Memory:** Stores temporary data during function execution.
- **Calldata:** Stores non-modifiable, non-persistent function input data, commonly used in external functions.

9. Transactions

Ether and Wei: Ether is the primary currency of Ethereum. Wei is its smallest unit.

1 Ether = 10^{18} Wei.

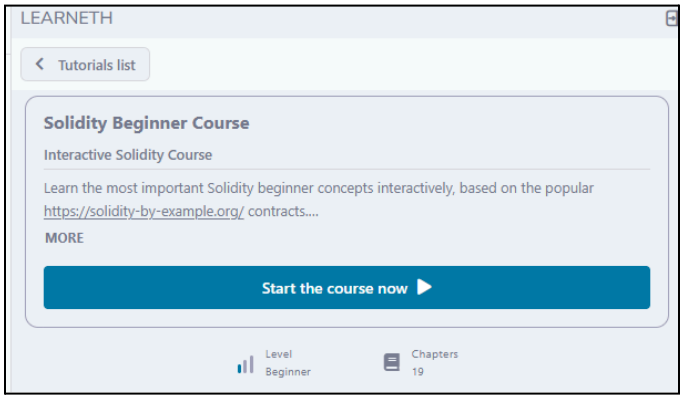
Gas and Gas Price: Gas represents the computational effort required to execute operations on the Ethereum network. Gas price determines the cost per unit of gas.

Sending Ether: Smart contracts can send Ether to other addresses using methods such as transfer(), send() and call().

Procedure / Steps Performed

Step 1: Open Remix IDE

1. Open the Remix IDE using https://remix.ethereum.org/.
2. Activate the LearnETH plugin through the Plugin Manager.
3. Open the Solidity Beginner Course.
4. Select the course containing 19 chapters.



Step 2: Configure Remix IDE

1. Open the Deploy & Run Transactions panel.
2. Select Remix VM as the execution environment.
3. Open the Solidity Compiler.

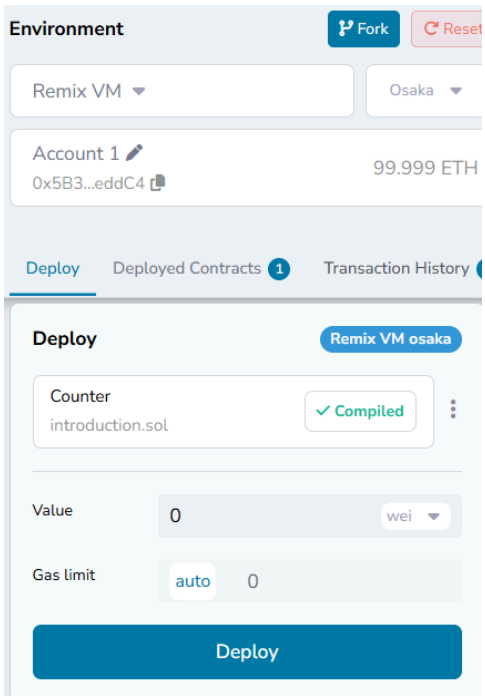
4. Select a compiler version compatible with the Solidity program.

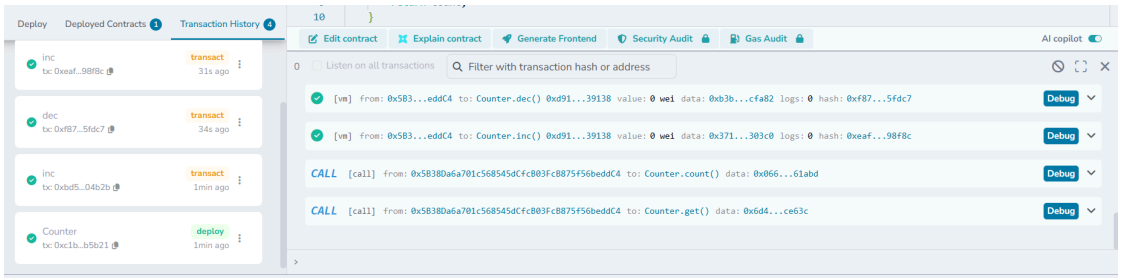
Step 3: Complete the 19 Course Chapters

1. Introduction

- Explored the introduction to Solidity and smart contracts.

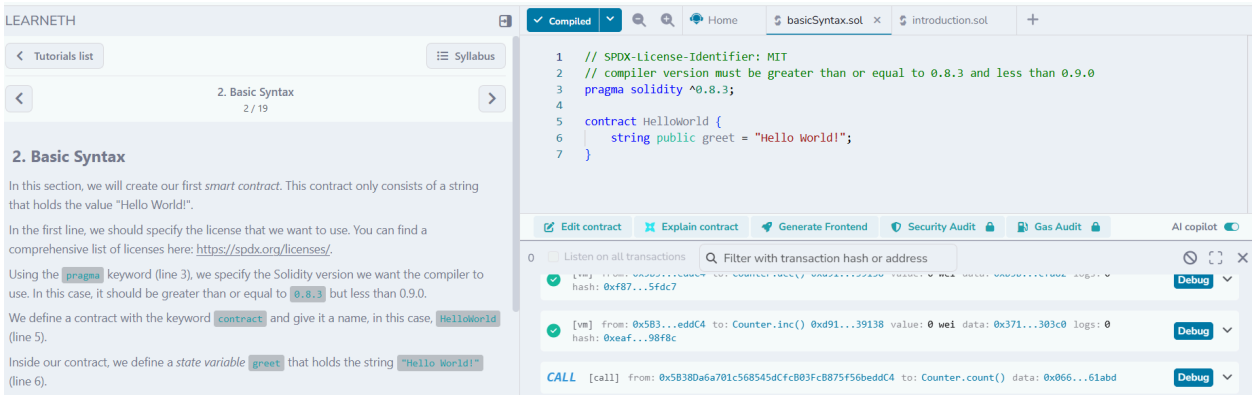






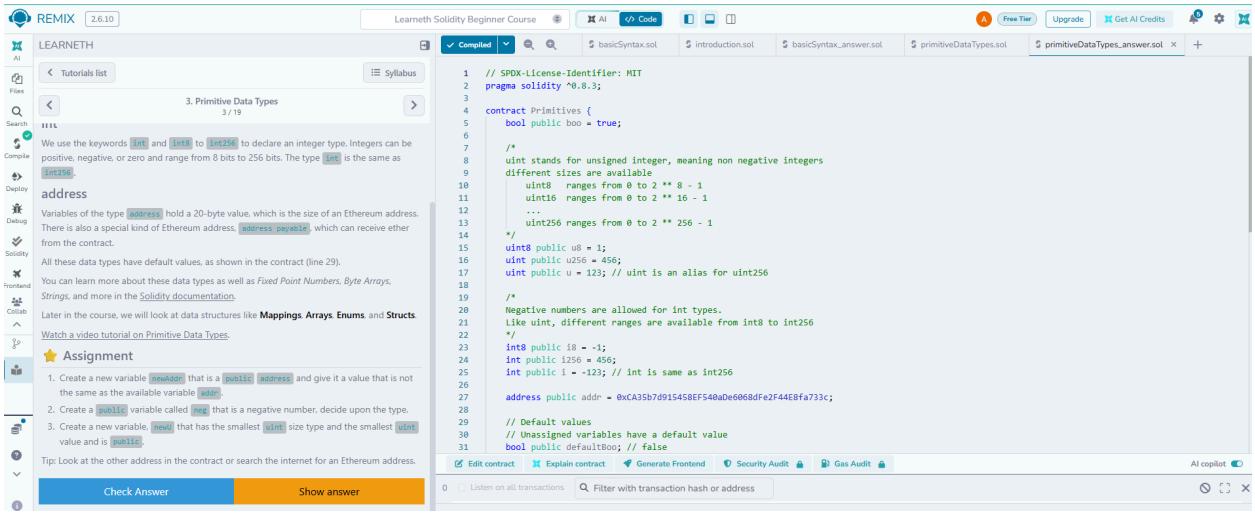
2. Basic Syntax

- Studied the basic structure and syntax of a Solidity program.



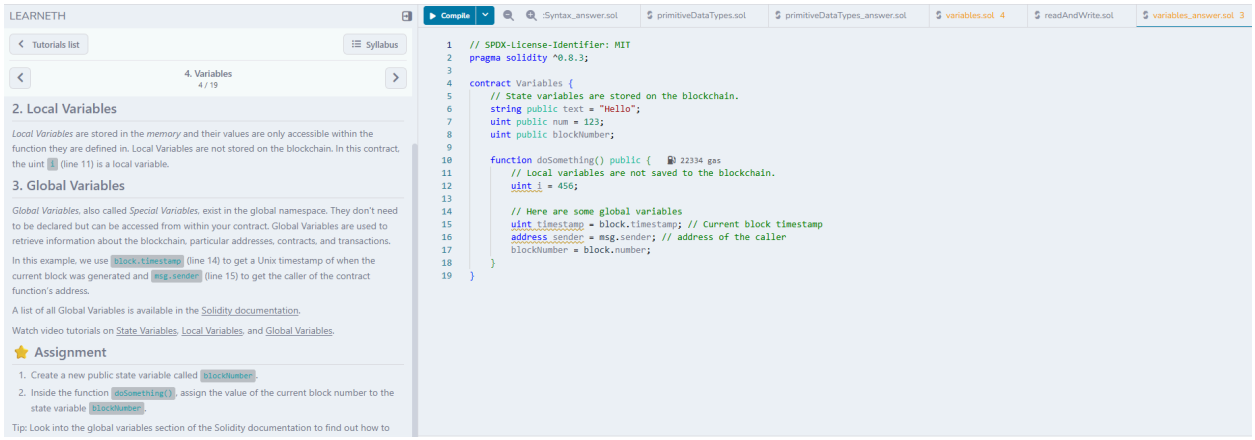
3. Primitive Data Types

- Explored primitive data types such as uint, int, bool, address, bytes and string.



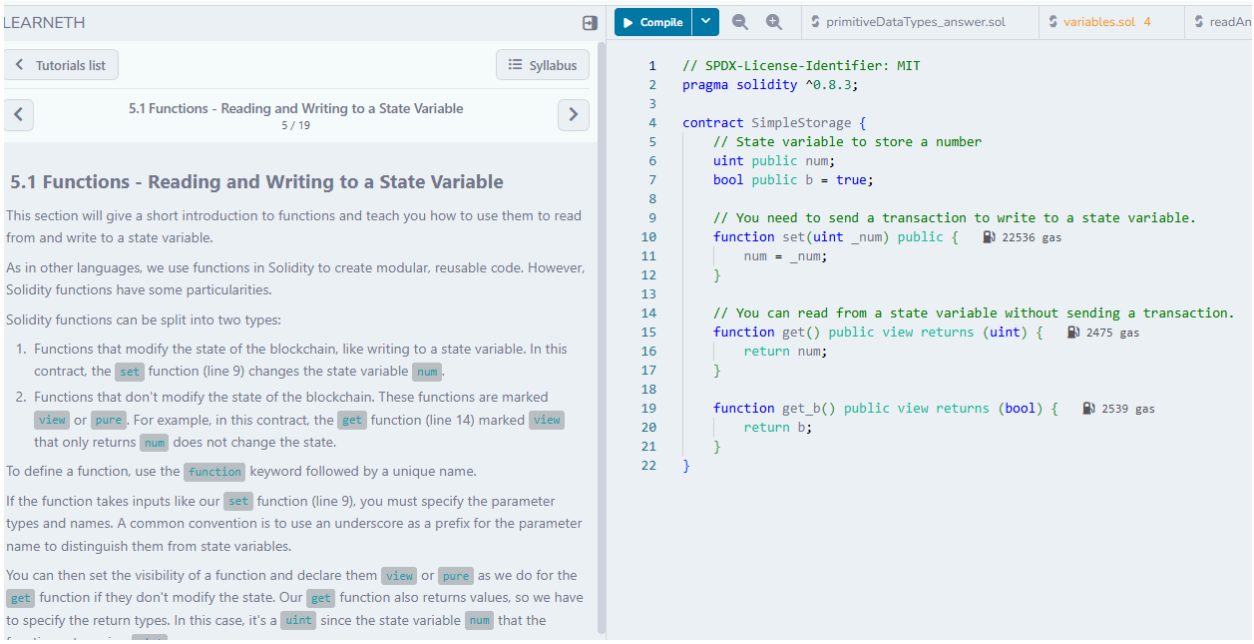
4. Variables

- Studied state variables, local variables and global variables.



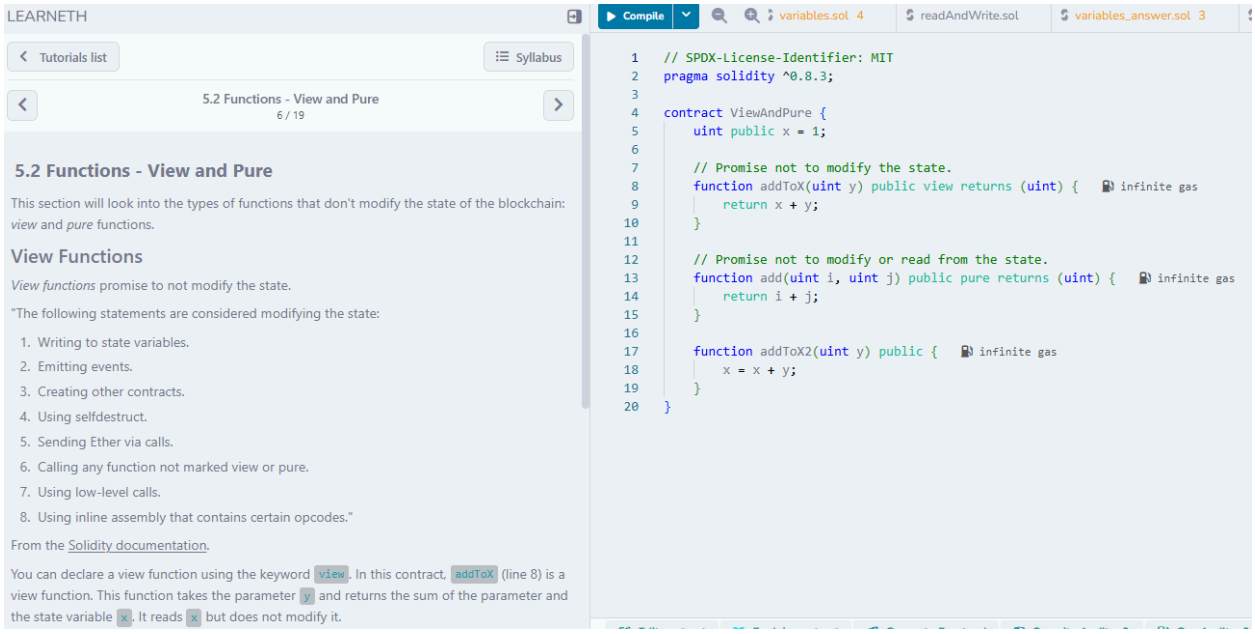
5.1 Functions - Reading and Writing to a State Variable

- Practised reading and modifying state variables using functions.



5.2 Functions - View and Pure

- Studied and implemented view and pure functions.



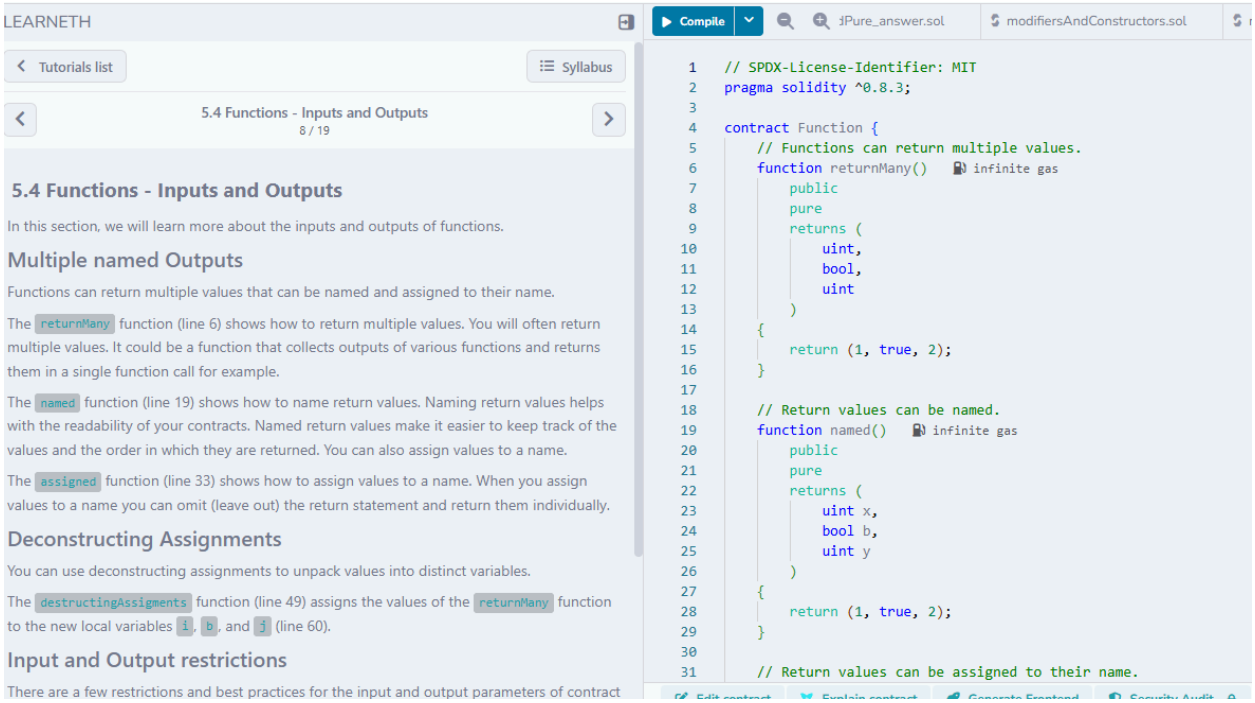
5.3 Functions - Modifiers and Constructors

- Explored function modifiers and constructors for initializing smart contracts and controlling function execution.



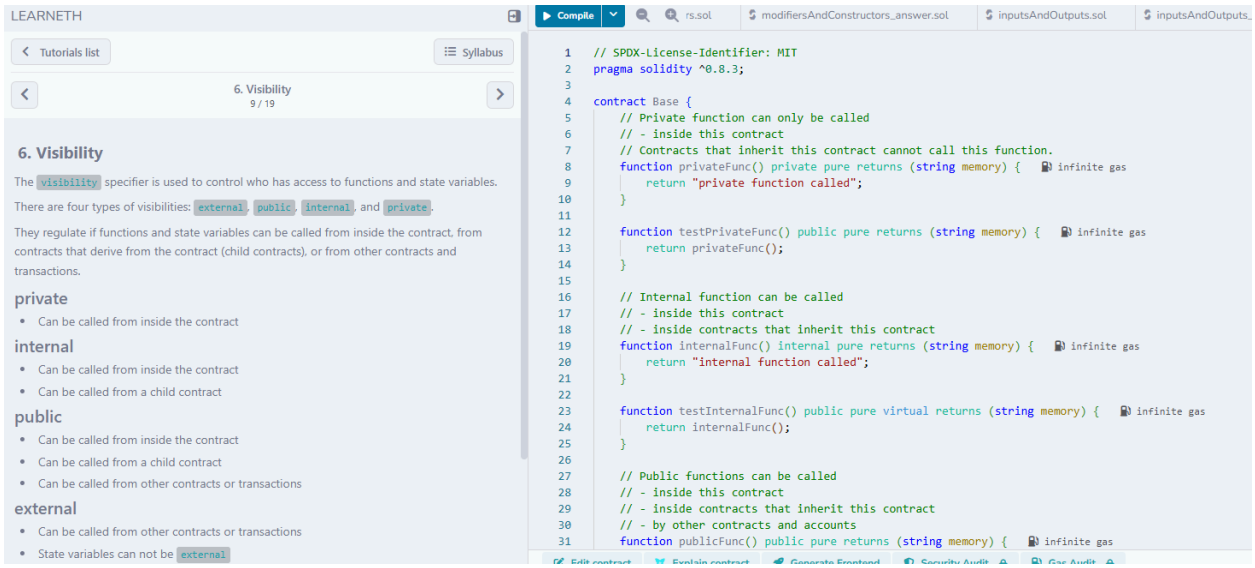
5.4 Functions - Inputs and Outputs

- Practised passing input parameters to functions and returning output values.



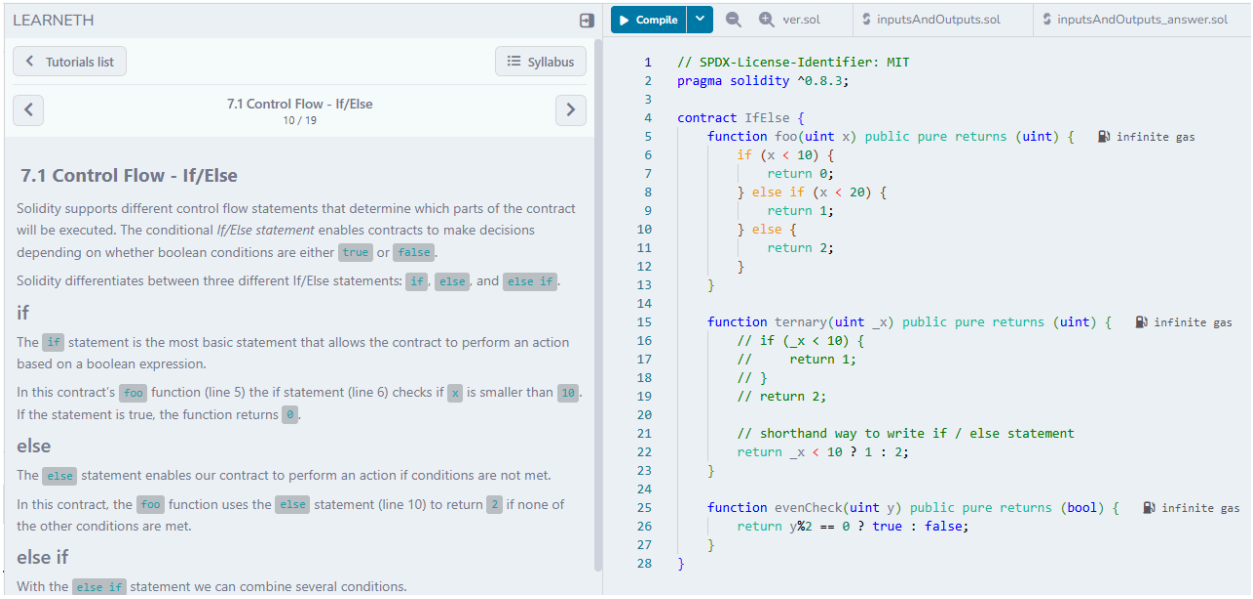
6. Visibility

- Studied public, private, internal and external visibility.



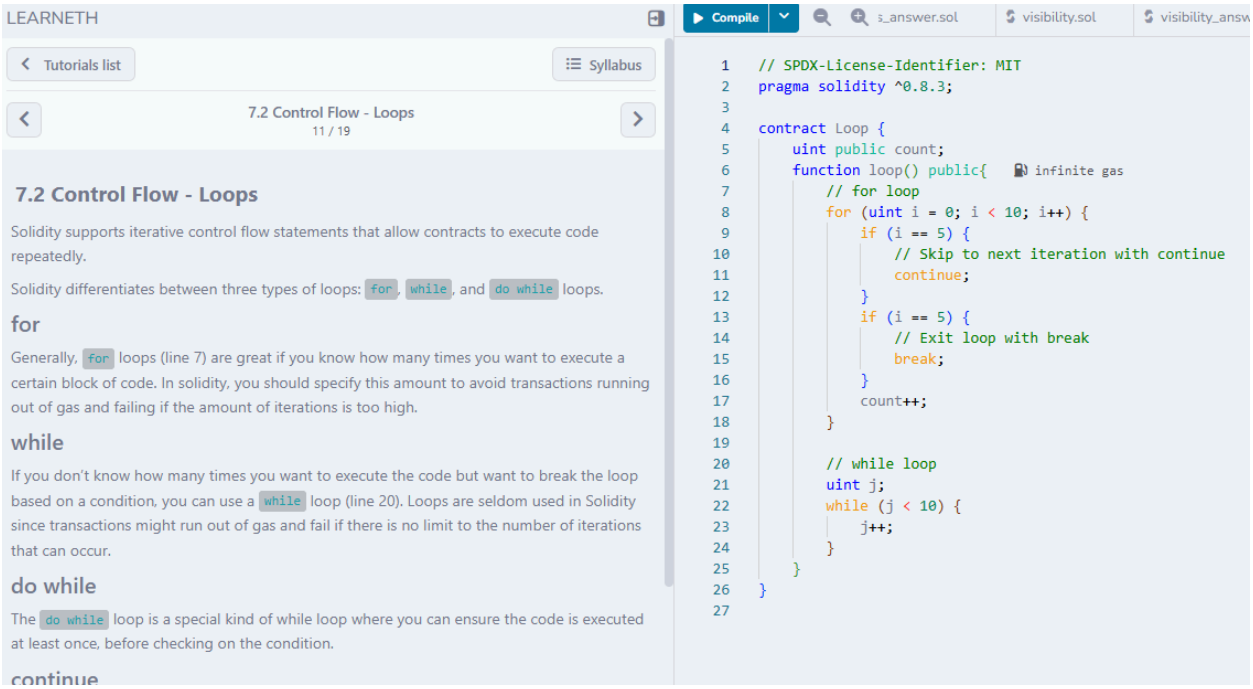
7.1 Control Flow - If/Else

- Implemented conditional statements using if-else.



7.2 Control Flow - Loops

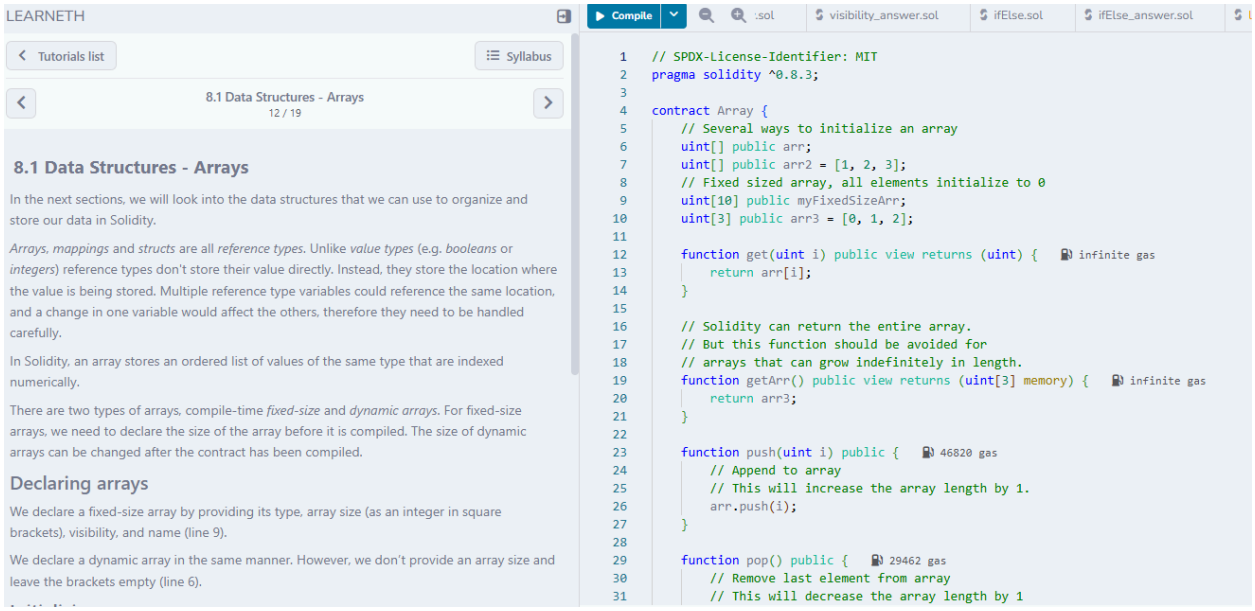
- Explored for, while and do-while loops for repeated execution



.

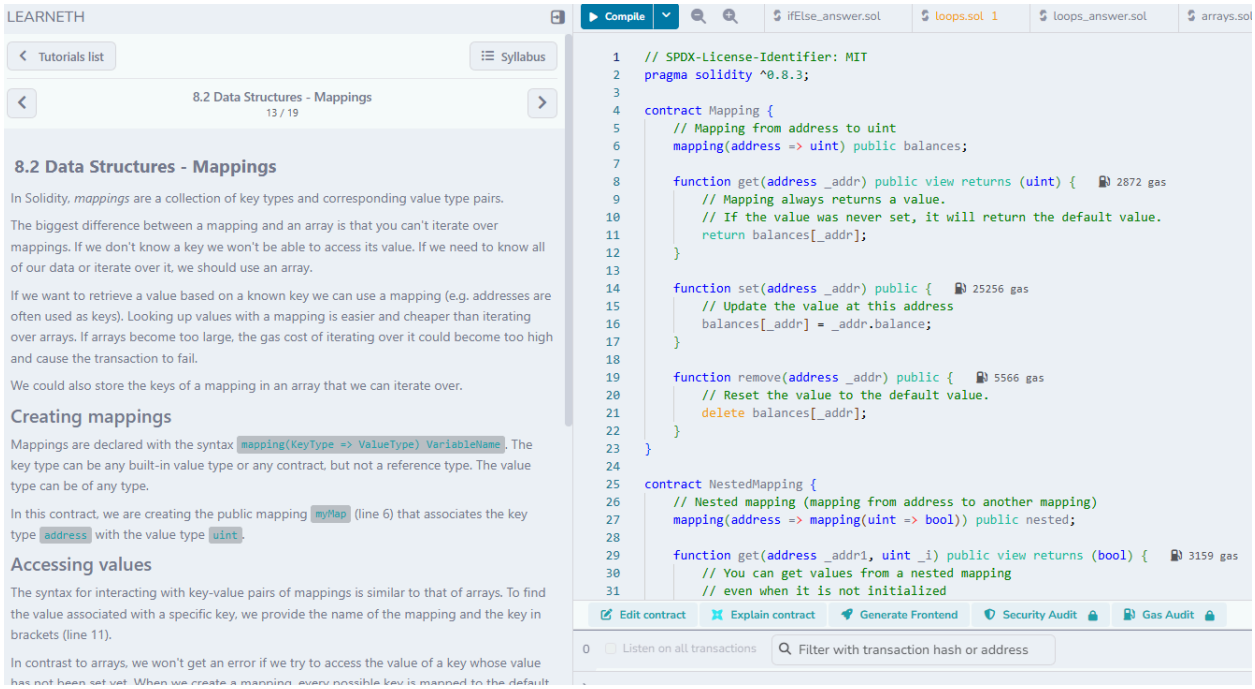
8.1 Data Structures - Arrays

- Practised storing and accessing elements using arrays.



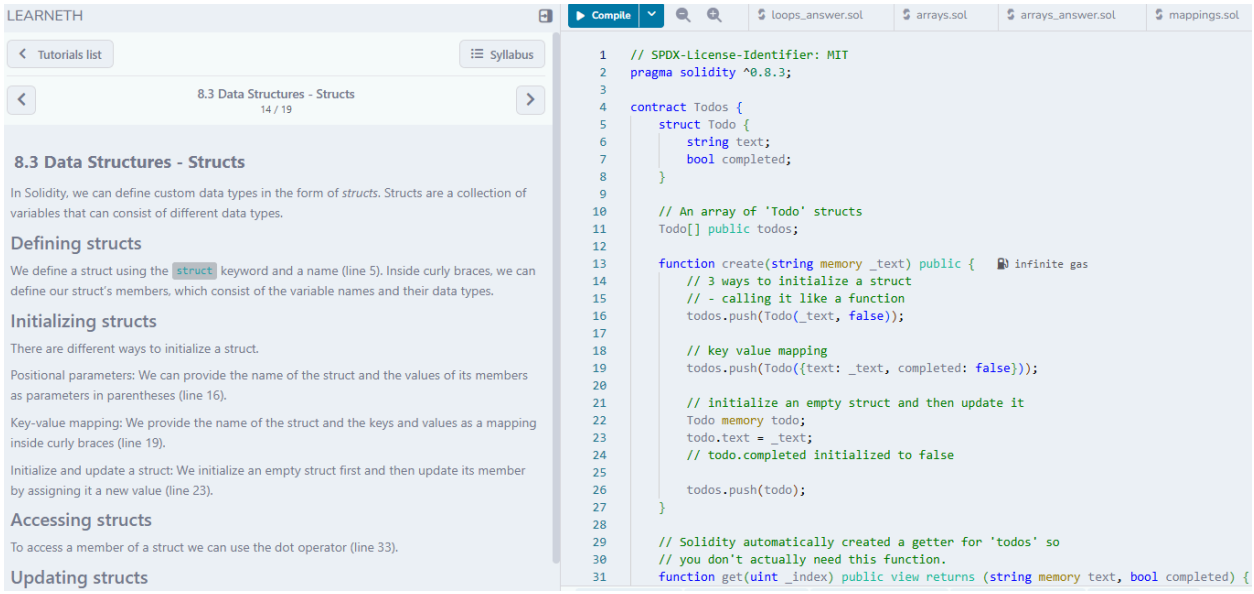
8.2 Data Structures - Mappings

- Studied key-value storage using mappings.



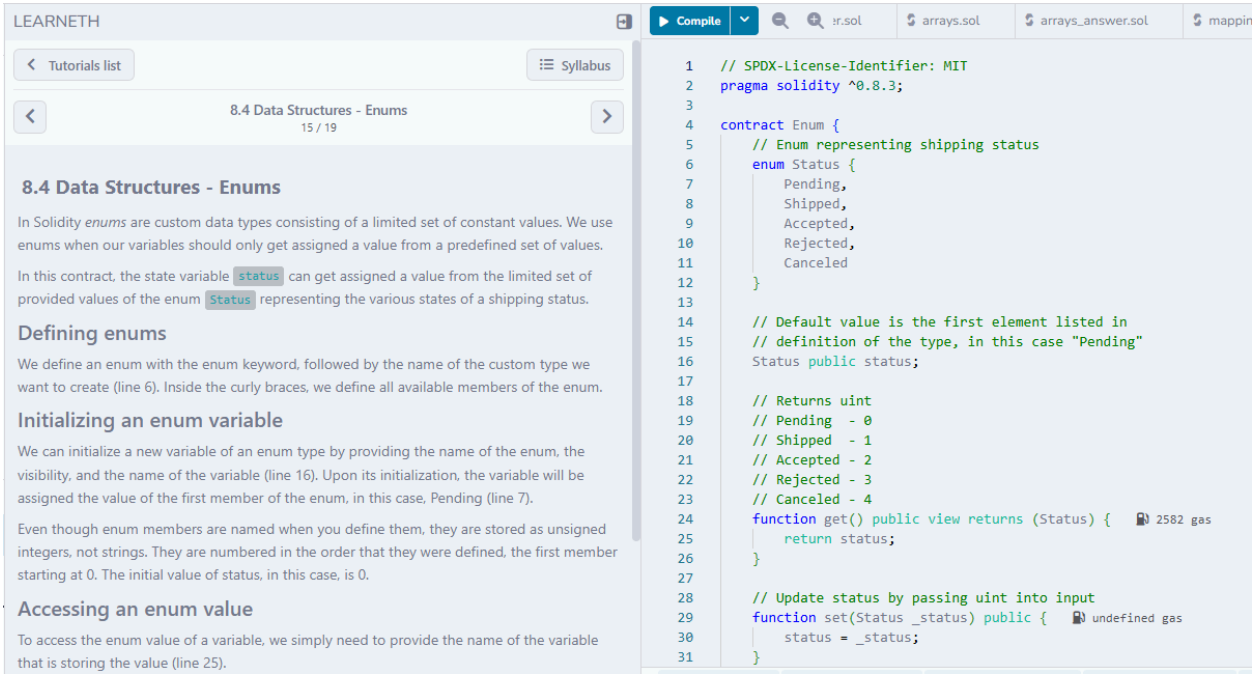
8.3 Data Structures - Structs

- Implemented structures to group related data.



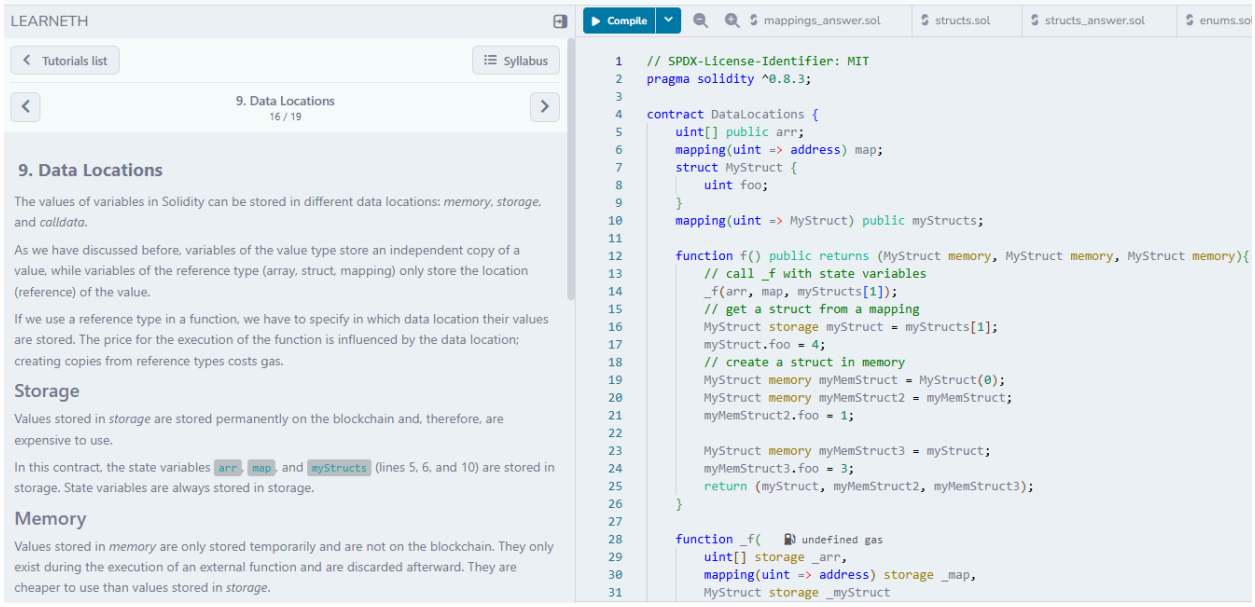
8.4 Data Structures - Enums

- Explored enumerations for defining predefined values.



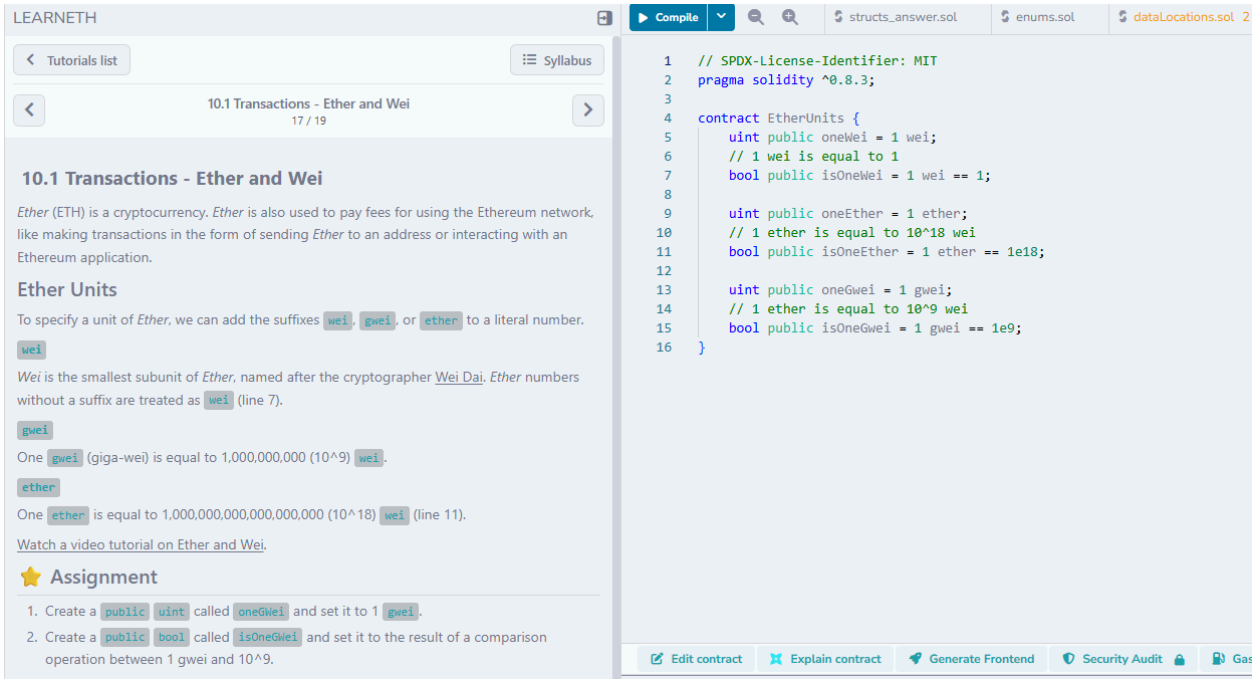
9. Data Locations

- Studied storage, memory and calldata.



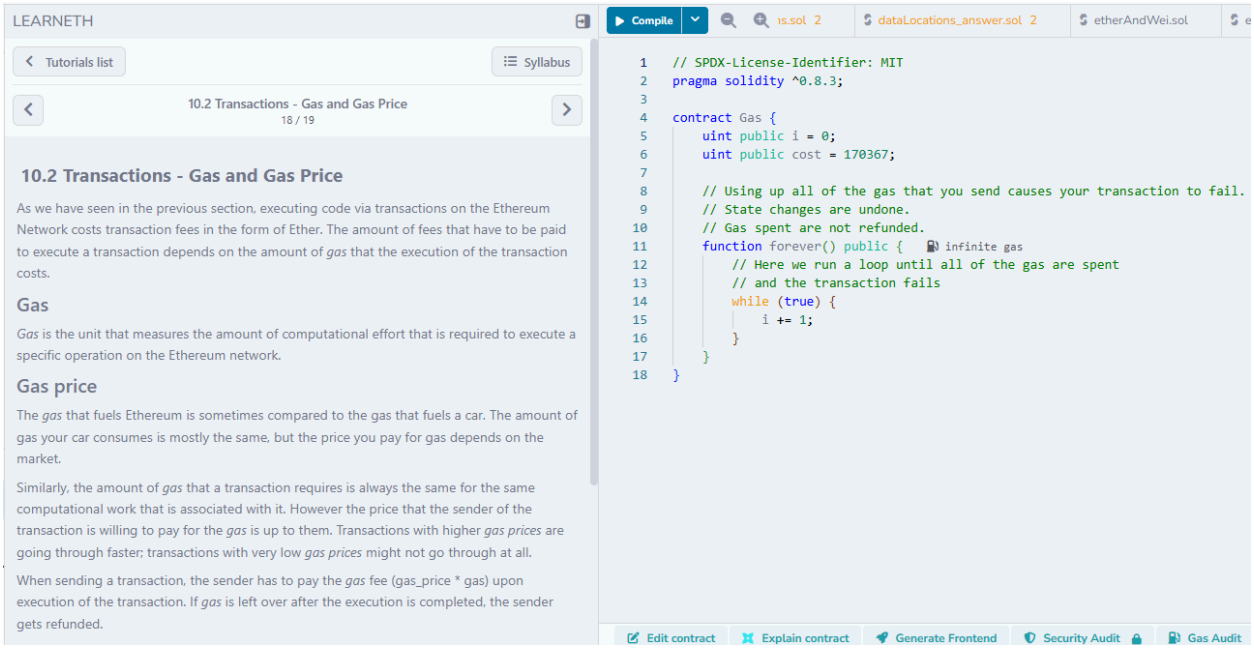
10.1 Transactions - Ether and Wei

- Understood Ether and Wei and their conversion.



10.2 Transactions - Gas and Gas Price

- Studied gas consumption and gas price in Ethereum transactions.



10.3 Transactions - Sending Ether

- Explored methods for sending Ether through smart contracts.



Conclusion

Through this experiment, the fundamentals of Solidity programming were explored by completing all 19 assignments provided in the Solidity Beginner Course. Concepts such as primitive data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, data locations and transactions were studied and practised. The experiment provided hands-on experience in writing, compiling, deploying and executing smart contracts using Remix IDE and Remix VM.